

BÁO CÁO TOÀN DIỆN THỰC NGHIỆM & ĐỊNH HƯỚNG NGHIÊN CỨU NCKH

Điều phối Chiến lược Suy luận Đa bước, Quyết định Dừng theo Ngân sách
và Đánh giá Khả năng Kháng Mã (Program Resistance) trên
Qwen3-8B

Nhóm Nghiên cứu Đề tài NCKH Sinh viên

Ngày cập nhật: 06 tháng 10 năm 2026

Abstract

Báo cáo này tổng hợp toàn bộ lộ trình nghiên cứu, dữ liệu thực nghiệm đã kiểm chứng, và phân tích cơ chế sâu sắc từ chuỗi chiến dịch khảo sát trên mô hình ngôn ngữ nhỏ (Qwen3-8B-4bit, MLX Apple Silicon). Trọng tâm nghiên cứu là giải quyết bài toán: *Khi nào mô hình cần suy luận từng bước, khi nào nên dừng/leo thang, tín hiệu tự tin nào đáng tin cậy, và ranh giới thực sự giữa suy luận ngôn ngữ tự nhiên (Chain-of-Thought / Tree-of-Thoughts) với việc sinh mã chương trình (Program-Aided Language) nằm ở đâu?*

Tài liệu cung cấp số liệu đối chiếu chi tiết từ 571 lượt sinh chuẩn (Run 11), 31 lượt đối chứng dẫn đạt Gap B, 100 lượt sinh PAL-v2 (Run 12), và 29 lượt sinh PAL-v3 (Run 13); chẩn đoán nguyên nhân thất bại ở cấp độ từng manh mối bài toán; giải mã nghịch lý chi phí của cơ chế cascade; và thiết lập 4 tiêu chí cốt lõi định nghĩa một họ bài toán “kháng giải bằng mã” (program-resistant) làm tiền đề xây dựng tập dữ liệu kiểm chứng độc lập (gen02_v2).

Contents

1	Bối cảnh Đề tài & Phương pháp Luận	2
1.1	Bối cảnh Khoa học & Vấn đề Nghiên cứu	2
1.2	Mô hình Khảo sát & Môi trường Thực thi	2
1.3	Không gian Chiến lược Suy luận (Strategy Arms)	2
1.4	Nguyên tắc Đăng ký Trước (Preregistration Rules)	3
2	Bảng Kết quả Thực nghiệm Toàn diện	4
2.1	Bảng Kết quả 6 Arms Chuẩn (Run 11, $N = 571$)	4
2.2	Bảng Kiểm định Mở rộng: PAL-v2 và PAL-v3 (Runs 12 & 13)	4
3	Phân tích Chuyên sâu theo Từng Họ Bài toán	6
3.1	Họ Số học (Arithmetic): Thống trị Tuyệt đối của Mã	6
3.2	Họ Xếp hạng Logic (Ordering): Điểm Hòa Hiệu năng ở 1/4 Chi phí	6
3.2.1	Chẩn đoán Bản chất 11 Ca Thất bại của PAL-v2 trên Ordering (AF3)	6
3.2.2	Bổ trợ Chéo giữa ToT và PAL-v2 (2x2 Contingency Table)	7
3.3	Họ Tổ hợp (Game-of-24): Thất bại của Negative Prompting	7
4	Phân tích Cơ chế & Kiểm định Tín hiệu	8
4.1	Đánh giá Cổng Preregistered Oracle: Sự Thật Đằng sau Con số 90.3%	8
4.2	Tại sao Oracle Quan sát (90.3%) < Oracle Độc lập (97.5%) Bác bỏ “Max trên Nhiều”?	8
4.2.1	Mô hình Xác suất Độc lập Lý thuyết	8
4.2.2	Ý nghĩa Thống kê	9
4.3	Bác bỏ Giả thuyết “Self-Consistency Hội tụ vào Đáp án Sai”	9
4.4	Nghịch lý Chi phí của Chính sách Leo thang Hậu nghiệm V'	9
5	Định nghĩa “Program-Resistant” & Định hướng NCKH	11
5.1	Khái niệm “Bài toán Kháng Mã” (Program-Resistant Benchmarks)	11
5.2	Lộ trình Triển khai cho Đề tài NCKH (Giai đoạn Confirmatory)	11
6	Bản kê Minh bạch & Dấu vết Kiểm toán (Audit Manifest)	13

Chapter 1

Bối cảnh Đề tài & Phương pháp Luận

1.1 Bối cảnh Khoa học & Vấn đề Nghiên cứu

Trong các mô hình ngôn ngữ lớn (LLM), việc kích hoạt chuỗi suy luận từng bước (*Chain-of-Thought* - CoT) hay tìm kiếm cây (*Tree-of-Thoughts* - ToT) thường mang lại bước nhảy vọt về độ chính xác. Tuy nhiên, trên các mô hình ngôn ngữ nhỏ (*Small Language Models* - SLM, $\leq 8B$ tham số), việc luôn luôn kích hoạt CoT mang lại hai bất lợi nghiêm trọng:

1. **Lãng phí ngân sách tính toán (Inference Compute Waste):** Đối với các bài toán đơn giản, CoT tiêu tốn gấp $3\times$ đến $10\times$ token mà không cải thiện độ chính xác, thậm chí làm tăng nguy cơ sinh lan man (wordiness) hoặc rơi vào bẫy lặp chậm trễ độ dài token (*length cutoff*).
2. **Thiếu tính trung thực (Faithfulness Gap):** Ở chế độ không bật bộ nhớ suy luận nội tại (`enable_thinking=False`), CoT chỉ là một đoạn giải trình hình thức được gọi mở bằng prompt (*elicited scratchpad*), không phải bằng chứng về chuỗi tư duy nhân quả nội tại của mô hình.

Đề tài NCKH này đặt mục tiêu xây dựng một chính sách điều phối động (*adaptive routing*) và dừng/leo thang theo ngân sách (*budget-aware optimal stopping*): khởi đầu bằng phương thức rẻ nhất, kiểm tra tính hợp lệ của chuỗi lời giải bằng các bộ kiểm định hình thức (*symbolic verifiers*), và chỉ leo thang sang các chiến lược tìm kiếm tốn kém hơn khi thực sự cần thiết.

1.2 Mô hình Khảo sát & Môi trường Thực thi

- **Mô hình nền tảng:** mlx-community/Qwen3-8B-4bit (Snapshot định danh cố định: 545dc4251c05440727734bc)
- **Cấu hình sinh:** `enable_thinking=False`, phần cứng Apple Silicon (MLX framework), bfloat16 sang NumPy an toàn.
- **Tập dữ liệu thăm dò:** `data/gen02_tune.json` ($N = 100$, development pool đã lộ) gồm 3 họ bài toán cấu trúc:
 - **Số học (Arithmetic, $N = 40$):** Chuỗi phép tính số học đa bước kết hợp nhiều toán tử.
 - **Xếp hạng logic (Ordering, $N = 31$):** Bài toán xếp thứ tự 5–7 nhân vật dựa trên các manh mối quan hệ trước/sau và khoảng cách (*gap clues*).
 - **Tổ hợp số học (Game-of-24, $N = 29$):** Tìm biểu thức số học từ 4 số cho trước để đạt kết quả đúng bằng 24.

1.3 Không gian Chiến lược Suy luận (Strategy Arms)

Nghiên cứu khảo sát và chuẩn hóa 7 chiến lược suy luận đại diện cho các trường phái tiếp cận:

1. **DIRECT-v2**: Yêu cầu mô hình trả lời trực tiếp đáp án cuối sau marker `Answer:`, không hiển thị các bước giải trung gian.
2. **COT (Elicited Scratchpad)**: Prompt yêu cầu viết các bước giải ngắn gọn không dùng mark-down/LaTeX và kết thúc bằng `Answer:`.
3. **SC (Self-Consistency)**: Lấy mẫu ngẫu nhiên độc lập $K = 5$ đường giải ở nhiệt độ $T = 0.7$, sau đó chọn đáp án bằng đa số phiếu (*majority voting*).
4. **TOT (Tree-of-Thoughts / Best-of-3 + Judge)**: Sinh 3 nhánh ứng viên độc lập ở $T = 0.7$, sau đó gọi một lượt prompt đánh giá (*judge*) với nhiệt độ $T = 0.0$ để chấm điểm và chọn ra nhánh tối ưu nhất.
5. **REACT**: Vòng lặp Thought-Action với công cụ ngoài `calculate[...]`. Mô hình chỉ sinh Thought và biểu thức tính toán; hệ thống tính toán và trả Observation cho tới khi mô hình gọi `finish[...]`.
6. **PAL (Program-Aided Language)**: Mô hình sinh một đoạn mã Python để giải bài toán, gán kết quả vào biến `result`, và được thực thi trong một môi trường sandbox an toàn.
7. **Policy V' (Cascade Escalation)**: Khởi chạy CoT $\rightarrow$ Dừng Verifier kiểm tra cú pháp và ràng buộc logic; nếu vi phạm hoặc chạm trần `length` $\rightarrow$ Leo thang sang ToT.

1.4 Nguyên tắc Đăng ký Trước (Preregistration Rules)

Để đảm bảo tính liêm chính học thuật và ngăn chặn việc "nhìn số liệu rồi mới đặt giả thuyết", nghiên cứu tuân thủ nghiêm ngặt các quy tắc đăng ký trước:

- **Cổng Quyết định Oracle (Decision Rule Oracle)**: Được đăng ký tại `prereg/decision_rule_oracle.md`. Một họ bài toán chỉ được phép chuyển sang giai đoạn fit router (**GO**) nếu:

$$\text{Gap} = \text{Oracle Acc} - \text{Best-Single Acc} \geq 10.0 \text{ pp} \quad \text{VÀ} \quad \text{Cl}_{\text{low}} \geq 5.0 \text{ pp}$$

trong đó khoảng tin cậy 95% được tính bằng phương pháp Cluster-Bootstrap theo `group_id` ($B = 2000$). Nếu không thỏa mãn, kết luận là **NO-GO**.

- **Tiêu chí Code-Solvable**: Được đăng ký tại `prereg/decision_rule_palv2.md` và `palv3.md`. Một họ bài toán được xác nhận là giải được hoàn toàn bằng mã nếu PAL đạt $\text{Acc} \geq \text{ToT}$ ở chi phí $\text{Tokens} \leq 0.5 \times \text{ToT}$. Khi đó các tuyên bố so sánh CoT/ToT trên họ đó bị rút lại.

Chapter 2

Bảng Kết quả Thực nghiệm Toàn diện

2.1 Bảng Kết quả 6 Arms Chuẩn (Run 11, $N = 571$)

Bảng dưới đây ghi nhận toàn bộ số liệu đo đạc thực tế từ trace `audit/run11_sweep_trace.jsonl`:

Họ Bài toán	Chiến lược (Arm)	n	Chính xác (corr/n)	Prompt Tok	Comp Tok	Tổng Tok	%Length
arith	DIRECT-v2	40	12.5% (5/40)	110.6	6.2	116.8	2.5%
	COT	40	75.0% (30/40)	125.6	180.0	305.6	0.0%
	SC ($K = 5$)	40	75.0% (30/40)	125.6	896.6	1022.2	0.0%
	TOT (Best-of-3)	40	75.0% (30/40)	125.6	563.0	688.6	0.0%
	REACT	40	82.5% (33/40)	86.6	102.1	188.7	0.0%
	PAL	40	100.0% (40/40)	131.6	62.5	194.1	0.0%
order	DIRECT-v2	31	22.6% (7/31)	102.1	6.1	108.1	0.0%
	COT	31	45.2% (14/31)	117.1	595.1	712.2	32.3%
	SC ($K = 5$)	31	35.5% (11/31)	117.1	2896.5	3013.6	48.4%
	TOT (Best-of-3)	31	67.7% (21/31)	117.1	1776.9	1894.0	0.0%
	REACT	31	19.4% (6/31)	78.1	56.3	134.3	0.0%
	PAL (cũ)	31	0.0% (0/31)	123.1	304.4	427.5	3.2%
g24	COT	29	48.3% (14/29)	78.0	549.8	627.8	44.8%
	SC ($K = 5$)	29	41.4% (12/29)	78.0	2487.0	2565.0	55.2%
	TOT (Best-of-3)	29	58.6% (17/29)	78.0	1368.8	1446.8	0.0%
	REACT	29	0.0% (0/29)	39.0	155.7	194.7	0.0%
	PAL (cũ)	29	0.0% (0/29)	84.0	398.1	482.1	27.6%

Table 2.1: Bảng tổng hợp hiệu năng 6 Arms trên tập development `gen02_tune` (Run 11).

2.2 Bảng Kiểm định Mở rộng: PAL-v2 và PAL-v3 (Runs 12 & 13)

Khi dỡ bỏ rào cản cấm `import` của `sandbox` (PAL-v2) và thêm gợi ý prompt tránh `eval` (PAL-v3), các kết quả chuyển biến mạnh mẽ:

Họ Bài toán	Arm	n	Chính xác (corr/n)	Prompt Tok	Comp Tok	Tổng Tok	%Length
arith	PAL	40	100.0% (40/40)	131.6	62.5	194.1	0.0%
	PAL-v2	40	100.0% (40/40)	154.6	74.6	229.1	0.0%
	TOT	40	75.0% (30/40)	125.6	563.0	688.6	0.0%
order	PAL (cấm import)	31	0.0% (0/31)	123.1	304.4	427.5	3.2%
	PAL-v2 (allowlist)	31	64.5% (20/31)	146.1	328.3	474.3	0.0%
	TOT	31	67.7% (21/31)	117.1	1776.9	1894.0	0.0%
g24	PAL (cũ)	29	0.0% (0/29)	84.0	398.1	482.1	27.6%
	PAL-v2 (chặn eval)	29	0.0% (0/29)	107.0	367.7	474.8	27.6%
	PAL-v3 (prompt no-eval)	29	0.0% (0/29)	128.0	398.9	526.9	20.7%
	TOT	29	58.6% (17/29)	78.0	1368.8	1446.8	0.0%

Table 2.2: Đối chiếu hiệu năng giữa các thể hệ PAL và TOT trên 3 họ bài toán.

Chapter 3

Phân tích Chuyên sâu theo Từng Họ Bài toán

3.1 Họ Số học (Arithmetic): Thống trị Tuyệt đối của Mã

Kết luận Then chốt trên Họ Số học

PAL đạt **40/40 (100.0%)** với chỉ **194.1 tokens**. ToT và CoT chỉ đạt 75.0% và tiêu tốn từ $1.5\times$ đến $3.5\times$ chi phí token. Không tồn tại khoảng trống đánh đổi (Trade-off gap = 0.0 pp) → ****NO-GO cho router****.

- **Cơ chế hoạt động:** Mô hình Qwen3-8B dịch hoàn hảo các biểu thức số học nhiều bước thành các phép gán biến Python tuần tự. Môi trường Python đảm bảo tính chính xác dấu chấm động và thứ tự ưu tiên phép toán, loại bỏ hoàn toàn các lỗi tích lũy làm tròn hay tính nhầm sai của LLM.
- **Thất bại của Cơ chế Cascade (Policy V):** Nếu khởi chạy CoT trước rồi mới leo thang sang PAL khi verifier phát hiện lỗi, hệ thống phải tiêu tốn trung bình 371.6 tokens ($1.91\times$ so với gọi PAL trực tiếp) mà độ chính xác không đổi (100%). Điều này chứng minh: trên các bài toán có công cụ thực thi mã rõ và chính xác tuyệt đối, việc áp dụng suy luận ngôn ngữ tự nhiên trước là một sự lãng phí tài nguyên thuần túy.

3.2 Họ Xếp hạng Logic (Ordering): Điểm Hòa Hiệu năng ở 1/4 Chi phí

Trước đây, Run 11 ghi nhận PAL đạt 0/31 trên Ordering, dẫn tới nhận định sai lầm rằng "ToT là quán quân suy luận duy nhất". Khi phân tích mã thô (AD4), phát hiện mô hình viết `from itertools import permutations` nhưng bị sandbox chặn từ khóa `import`. Khi mở allowlist trong PAL-v2:

- **Hiệu năng bùng nổ:** PAL-v2 nhảy vọt từ 0% lên **64.5% (20/31 đúng)**, chỉ kém ToT đúng 1 câu duy nhất (20 vs 21 câu) nhưng tiêu tốn chỉ **474.3 tokens** so với **1894.0 tokens** của ToT (tiết kiệm tới **75% ngân sách token**).

3.2.1 Chẩn đoán Bản chất 11 Ca Thất bại của PAL-v2 trên Ordering (AF3)

Phân tích mã nguồn của 11 câu sai trong `audit/AF_order_palv2_failed_clues.txt` chỉ ra hai nguyên nhân lỗi hệ thống:

1. **Dịch sai offset khoảng cách (5 ca: order_0008, 0010, 0017, 0028, 0029):** Mạnh mối đề bài ghi: *"Erin finished exactly 2 places ahead of Dave, with exactly 1 runner between them"*. Mô hình hiểu nhầm và viết vào code:


```
if erin_idx - dave_idx == 3: # Dừng khoảng cách 3 thay vì 2!
```

hoặc dùng hàm không hướng `abs(pos1 - pos2) != 3`. Đây thuần túy là **lỗi đọc hiểu ngôn ngữ tự nhiên**, không phải lỗi thuật toán duyệt hoán vị.

2. **Ràng buộc mâu thuẫn nội tại (5 ca: order_0003, 0019, 0022, 0023, 0031):** Mô hình dịch các manh mối thành các điều kiện `if` loại trừ lẫn nhau, dẫn đến việc duyệt hết toàn bộ $N!$ hoán vị mà không có hoán vị nào thỏa mãn, vòng lặp kết thúc mà không gán giá trị cho `result` (`ValueError: code produced no result`).

3.2.2 Bỏ trợ Chéo giữa ToT và PAL-v2 (2x2 Contingency Table)

Bảng kiểm định tương quan cặp (AF2) cho thấy tính bù trừ sâu sắc giữa hai cơ chế tư duy:

- **Cả hai cùng đúng:** 16 câu (51.6%).
- **Chỉ PAL-v2 đúng:** 4 câu (12.9%) — những câu có chuỗi quan hệ phức tạp khiến ToT bị quá tải context hoặc kẹt trần token, nhưng thuật toán duyệt vết cặn Python xử lý trong 2 ms.
- **Chỉ ToT đúng:** 5 câu (16.1%) — những câu mà PAL-v2 dịch nhầm offset khoảng cách ($k \rightarrow k+1$), trong khi khả năng đọc hiểu ngôn ngữ tự nhiên của ToT đã giúp nó diễn giải đúng mỗi quan hệ.
- **Cả hai cùng sai:** 6 câu (19.4%) — các bài toán có mật độ ràng buộc đa tầng thực sự khó.
- **Tỷ lệ bất đồng (Discordant Rate):** Đạt tới **29.0% (9/31 câu)**.

3.3 Họ Tổ hợp (Game-of-24): Thất bại của Negative Prompting

Bản chất Thất bại trên Game-of-24

Cả PAL, PAL-v2 và PAL-v3 đều đạt **0/29 (0.0%)** trên Game-of-24. ToT là phương pháp sinh tự nhiên duy nhất giải quyết được họ bài toán này (**58.6%, 17/29 đúng**).

- **Artifact ở PAL và PAL-v2:** Ở Run 11, mô hình bị chặn import. Ở Run 12 (PAL-v2), mô hình tự động sinh hàm `def evaluate(expr): return eval(expr)` và bị sandbox chặn từ khóa `eval` vì lý do an ninh.
- **Kiểm định quyết định PAL-v3 (Run 13):** Trong Prompt AF1, hệ thống đã đưa ra chỉ dẫn phủ định tường minh: *"eval, exec, compile are unavailable; carry (value, expression-string) pairs and build expressions recursively"*.
- **Kết quả thực tế: 29/29 bài mô hình vẫn tiếp tục sinh hàm gọi eval!** Thiên kiến huấn luyện (*pretraining prior*) của Qwen3-8B đối với bài toán Game-of-24 trong Python gắn chặt với thư viện `eval`. Mô hình 8B không đủ năng lực tuân thủ chỉ dẫn phủ định phức tạp để tự viết một bộ duyệt cây biểu thức đệ quy kèm phân số trong giới hạn 512 token.
- **Ý nghĩa học thuật:** Game-of-24 là một minh chứng thực nghiệm điển hình về việc: *Khi tước bỏ công cụ thực thi động bên ngoài, bài toán trở thành kháng mã đối với SLM, và tìm kiếm cây trên ngôn ngữ tự nhiên (ToT) tái khẳng định ưu thế vượt trội.*

Chapter 4

Phân tích Cơ chế & Kiểm định Tín hiệu

4.1 Đánh giá Cổng Preregistered Oracle: Sự Thật Đằng sau Con số 90.3%

Trong Prompt AE, khi bổ sung PAL-v2 thành arm thứ 7, Oracle trên họ Ordering ghi nhận bước nhảy vọt lên **90.3% (28/31 đúng)**, **Gap = +22.6 pp**, **CI [+9.7 pp; +38.7 pp]** và được hệ thống báo **GO**. Tuy nhiên, phân tích sâu ở AF2 đã bóc tách bản chất của con số này:

Tập Hợp Arm Khảo sát	Best-Single Arm	Oracle Accuracy	Oracle Tokens	Gap (pp)	95% Cluster-Boot
7 Arms đầy đủ (AE)	TOT (67.7%)	90.3% (28/31)	509.1	+22.6 pp	[+9.7 pp; +38.7 pp]
{COT, TOT, PAL-v2} (AF2)	TOT (67.7%)	83.9% (26/31)	695.5	+16.1 pp	[+3.2 pp; +29.0 pp]
{TOT, PAL-v2} (AF2)	TOT (67.7%)	80.6% (25/31)	703.0	+12.9 pp	[+3.2 pp; +25.8 pp]

Table 4.1: Đối chiếu Oracle khi loại bỏ các arm đoán mò (DIRECT-v2, REACT) trên họ Ordering.

Nhận định Khoa học về Cổng Oracle

Khi loại bỏ hai arm ở mức đoán mò là DIRECT-v2 (22.6%) và REACT (19.4%), Oracle thực chất trên các arm mạnh chỉ đạt **80.6% đến 83.9%**. Cả hai tập hợp này đều có **cận dưới khoảng tin cậy 95% rơi xuống +3.2 pp (< 5.0 pp)**, do đó theo đúng tiêu chí khắt khe của preregistration, cổng Ordering **chưa đạt điều kiện GO**.

Con số 90.3% trước đó được hưởng lợi từ việc DIRECT-v2 đoán ngẫu nhiên trúng 2 câu (order_0010 và 0031) mà cả ba arm mạnh đều giải sai. Việc kiểm định lại trên tập arm mạnh đã ngăn chặn kết luận sai lầm do hiệu ứng cộng dồn may rủi.

4.2 Tại sao Oracle Quan sát (90.3%) < Oracle Độc lập (97.5%) Bác bỏ “Max trên Nhiều”?

4.2.1 Mô hình Xác suất Độc lập Lý thuyết

Nếu 7 arm có xác suất giải đúng độc lập với nhau trên từng câu hỏi ($p_1, p_2, \dots, p_7$), xác suất để ít nhất một arm giải đúng bài toán là:

$$P(\text{Oracle Độc lập}) = 1 - \prod_{k=1}^7 (1 - p_k)$$

Thay các xác suất thực tế trên tập Ordering ($N = 31$):

$$\prod_{k=1}^7 (1 - p_k) = (1 - 0.226)(1 - 0.452)(1 - 0.355)(1 - 0.677)(1 - 0.194)(1 - 0.0)(1 - 0.645) \approx 0.0252$$

$$\implies P(\text{Oracle Độc lập}) = 1 - 0.0252 = \mathbf{97.5\%}$$

4.2.2 Ý nghĩa Thống kê

- **Hiện tượng Max trên Nhiễu (Optimization on Noise):** Xảy ra khi ta gộp nhiều phép thử ngẫu nhiên yếu; giá trị cực đại quan sát được sẽ tự động bị thổi phồng tiến sát về kỳ vọng độc lập (97.5%) do hiện tượng tích lũy sai số loại I.
- **Bằng chứng về Lỗi Tương quan (Correlated Errors):** Con số thực tế đạt 90.3% (thấp hơn đáng kể mức 97.5%) chứng minh rằng các arm **không thất bại ngẫu nhiên**. Có những bài toán có độ phức tạp khách quan cao khiến mọi mô hình đều vấp ngã.
- **Tính Bỏ trợ Thực chất:** Khoảng cách +22.6 pp (hay +12.9 pp trên arm mạnh) phản ánh sự bù trừ thực chất giữa hai không gian biểu diễn (ngôn ngữ tự nhiên của ToT vs cú pháp hình thức của PAL-v2), chứ không phải do chọn lọc cực đại trên nhiễu.

4.3 Bác bỏ Giả thuyết “Self-Consistency Hội tụ vào Đáp án Sai”

Trước đây từng có giả định cho rằng Self-Consistency (SC) suy giảm trên các bài toán logic vì mô hình bị củng cố thiên kiến sai lầm (*bias reinforcement*). Phân tích phân phối tỷ lệ phiếu chiến thắng (Winner Vote Share $k/5$, lưu tại `audit/AD_sc_consensus.txt`) đã trực tiếp bác bỏ điều này:

Họ Bài toán	Phân nhóm	n	Mean Winner Share	Median Share	Số ca có Winner Share ≥ 0.6 ($\geq 3/5$ phiếu)
order	Lời giải Đúng	11	0.782	0.800	10 / 11 (90.9%)
	Lời giải Sai	20	0.440	0.400	6 / 20 (30.0%)
g24	Lời giải Đúng	12	0.600	0.600	8 / 12 (66.7%)
	Lời giải Sai	17	0.282	0.200	1 / 17 (5.9%)

Table 4.2: Phân phối tỷ lệ phiếu bầu của nhánh chiến thắng trong Self-Consistency ($K = 5$).

Kết luận: Khi SC đưa ra đáp án sai, mức đồng thuận trung bình chỉ là 0.440 (Order) và 0.282 (G24). Trên G24, median winner share chỉ là 0.20 (nghĩa là 5 mẫu sinh ra 5 đáp án sai hoàn toàn khác nhau). Lỗi của SC là **sự phân tán phiếu (dispersion)** trong không gian tìm kiếm rộng lớn, chứ không phải do các mẫu cùng đồng thuận vào một điểm sai cục bộ.

4.4 Nghịch lý Chi phí của Chính sách Leo thang Hậu nghiệm V'

Chính sách V' (chạy CoT $\rightarrow$ Verifier cờ báo lỗi $\rightarrow$ Leo thang ToT) được thử nghiệm tại `audit/AD_posthoc_tot.txt`.

- **Ordering:** V' đạt 71.0% (22/31) so với ToT đạt 67.7% (21/31) $\rightarrow$ Tăng vón vẹn 1 câu đúng. Nhưng chi phí token là **2088.1 tokens vs 1894.0 tokens (đắt hơn 10.2%)**.
- **Game-of-24:** V' đạt 62.1% (18/29) so với ToT đạt 58.6% (17/29) $\rightarrow$ Tăng 1 câu đúng. Nhưng chi phí token là **1649.8 tokens vs 1446.8 tokens (đắt hơn 14.0%)**.

Nguyên nhân Kinh tế Tính toán của Nghịch lý V'

Về mặt toán học, một cơ chế cascade chỉ tiết kiệm chi phí so với always-escalate khi tỷ lệ leo thang thỏa mãn:

$$P(\text{escalate}) < 1 - \frac{c_{\text{COT}}}{c_{\text{TOT}}} \approx 1 - \frac{712}{1894} \approx 62.4\%$$

Trên thực tế, tỷ lệ leo thang của Verifier lên tới **64.5%** (20/31 câu bị cờ báo lỗi). Hơn nữa, những bài toán bị đẩy sang ToT là những bài toán phức tạp, khiến lượt sinh ToT phía sau tiêu tốn nhiều token hơn mức trung bình. Do đó, V' bị lỗ vốn về chi phí và vi phạm tiêu chí preregistration ($\text{Cost} \leq 0.6 \times E$).

Chapter 5

Định nghĩa “Program-Resistant” & Định hướng NCKH

5.1 Khái niệm “Bài toán Kháng Mã” (Program-Resistant Benchmarks)

Qua chuỗi thí nghiệm từ PAL-v1 đến PAL-v3, một đóng góp lý luận quan trọng của đề tài là làm sáng tỏ ranh giới: *Khi nào một bài toán được coi là giải được bằng mã (code-solvable), và khi nào mã không phải là lời giải miễn phí (program-resistant)?*

Một họ bài toán được xác định là **Program-Resistant** khi thỏa mãn ít nhất 2 trong 4 tiêu chí cốt lõi sau:

1. **Ràng buộc phụ thuộc Ngữ nghĩa Mềm (Soft / Semantic Constraints):** Trong số học hay xếp hạng, không gian ràng buộc mang tính hình thức tuyệt đối ($x > y$, $a + b = c$). Trái lại, nếu bài toán đòi hỏi đánh giá các thuộc tính ngôn ngữ, suy luận phản thực tế (*counterfactual reasoning*), hoặc đánh giá hành vi hợp lý trong ngữ cảnh mở, một đoạn mã Python thuần túy không thể biểu diễn bằng các câu lệnh `if/else`. Để viết script giải bài này, chính đoạn code đó lại phải gọi API của LLM để đánh giá từng bước → Code trở thành một lớp vỏ bọc dư thừa.
2. **Bùng nổ Không gian Trạng thái Vượt trần Sandbox (Combinatorial Explosion > 5s):** Với bài toán xếp hạng 5–6 người, số hoán vị chỉ là $5! = 120$ đến 720, Python vét cạn trong vài mili-giây. Nhưng nếu bài toán quy hoạch có không gian trạng thái 10^{15} (ví dụ: bài toán định tuyến ràng buộc thời gian hoặc lập lịch dự án nhiều chiều), thuật toán brute-force sẽ bị timeout ngay lập tức trong sandbox (ngưỡng 5 giây). Để giải được bằng mã, mô hình bắt buộc phải tự thiết kế một giải thuật heuristic tối ưu phức tạp (A^* , Branch & Bound)—điều mà một SLM 8B trong phạm vi 512 tokens sinh mã gần như không thể làm đúng.
3. **Môi trường Động Thiếu Khép kín (Non-closed World / Interactive Grounding):** PAL chỉ phát huy tác dụng khi đề bài cung cấp trọn vẹn dữ kiện đầu vào để mô hình viết một script chạy một lèo từ đầu đến cuối. Nếu bài toán đòi hỏi khám phá thông tin theo từng bước tương tác (multi-hop retrieval, tra cứu tài liệu theo ngữ cảnh như ReAct), một đoạn mã tĩnh không thể biết trước kết quả tra cứu của bước sau để hardcode logic.
4. **Rào cản Cấu trúc Hình thức & AST (Inversion / AST Complexity):** Diễn hình như Game-of-24: khi tước bỏ hàm thực thi chuỗi động `eval()`, mô hình bắt buộc phải tự dựng một mini-compiler, duyệt cây biểu thức hoặc hàm đệ quy sinh giá trị phân số. Độ phức tạp biểu diễn này vượt quá dung lượng working memory của SLM 8B, khiến việc sinh mã thất bại hoàn toàn.

5.2 Lộ trình Triển khai cho Đề tài NCKH (Giai đoạn Confirmatory)

Dựa trên các phát hiện thực nghiệm đã được kiểm chứng độc lập, nhóm nghiên cứu đề xuất lộ trình hành động cụ thể cho giai đoạn tiếp theo:

1. **Bổ sung Họ Bài toán Program-Resistant vào Bộ Sinh dữ liệu:** Trước khi mở tập kiểm chứng chính thức (`gen02_v2.json`), cần bổ sung một họ bài toán mà chương trình Python không phải là lời giải miễn phí, ví dụ: *Suy luận logic ngữ nghĩa đa bước trên đoạn văn diễn đạt lại (Multi-hop Defeasible Reasoning with Natural Language Clues)*. Đây là môi trường mà CoT và ToT thực sự phát huy giá trị tìm kiếm ngữ nghĩa mà PAL không thể thay thế.

2. **Chuyển trọng tâm Routing sang Cơ chế Phân quyền Lai (Hybrid Delegation):** Thay vì so sánh thuần túy giữa các arm sinh văn bản (DIRECT vs CoT vs ToT), router sẽ giải quyết bài toán phân quyền:

Đầu vào $\rightarrow \begin{cases} \text{Bài toán số học/hình thức thuần} & \rightarrow \text{Ủy quyền cho Python (PAL-v2)} \\ \text{Bài toán ngữ nghĩa/không gian tìm kiếm mở} & \rightarrow \text{Suy luận nội tại (CoT / ToT)} \end{cases}$

3. **Chạy Kiểm định Độc lập Đa Hạt giống trên `gen02_v2`:** Thực hiện kiểm định xác nhận (*confirmatory evaluation*) với các seed độc lập, đồng bằng hoàn toàn các ngưỡng quyết định và quy tắc verifier đã fit trên tập `tune`, bảo đảm tính khách quan tuyệt đối cho bài báo khoa học.

Chapter 6

Bản kê Minh bạch & Dấu vết Kiểm toán (Audit Manifest)

Nhằm bảo đảm tính tái lập hoàn toàn (*full reproducibility*) cho công trình NCKH, toàn bộ các lượt sinh thực nghiệm, mã nguồn và dữ liệu đều được định danh bằng mã băm SHA-256 và lưu vết tại kho lưu trữ Git:

Tệp Dấu vết Thực nghiệm	Git Tag	Số bản ghi	Mã băm SHA-256	Ghi chú Kiểm toán
audit/run11_sweep_trace.jsonl	run11	571	3d576747	6 Arms chuẩn trên Qwen3-8B
audit/gapB_sweep_trace.jsonl	gapB	31	a6c5201c	Đổi chứng diễn đạt khoảng cách Style B
audit/ae_palv2_trace.jsonl	run12	100	c3f2500e	Sweep.PAL-v2 sandbox allowlist
audit/af_palv3_trace.jsonl	run13	29	4967500e	Sweep.PAL-v3 negative prompt trên G24

Table 6.1: Bản kê dữ liệu thực nghiệm đã kiểm toán và lưu vết bất biến.

• Các tệp văn bản kiểm toán nguồn (Raw Artifacts):

- audit/AD_report_results.txt: Bảng mô tả 6 arms chuẩn.
- audit/AD_oracle_rule.txt: Đánh giá cổng preregistered oracle 6 arms.
- audit/AD_posthoc_tot.txt: Số liệu chi tiết chính sách leo thang V' .
- audit/AD_sc_consensus.txt: Phân tích phân phối vote share của Self-Consistency.
- audit/AE_table_pal_palv2_tot.txt: Bảng so sánh 3 thể hệ PAL và ToT.
- audit/AE_oracle_rule_7arm.txt: Đánh giá cổng oracle 7 arms.
- audit/AF_strong_arms_oracle.txt: Phân tích oracle trên các tập arm mạnh và bảng tương quan 2×2 .
- audit/AF_order_palv2_failed_clues.txt: Bảng chẩn đoán chi tiết 11 câu Ordering thất bại của PAL-v2.
- audit/AF_palv3_verdict.txt: Phán quyết chính thức cho PAL-v3 trên Game-of-24.

- **Cơ chế an toàn cấu hình:** URL push của remote origin đã được chuyển sang chế độ DISABLED ở tầng Git config, ngăn chặn hoàn toàn việc rò rỉ mã nguồn hoặc kết quả chưa qua duyệt.

— HẾT BÁO CÁO —

Tài liệu biên soạn phục vụ công tác nghiên cứu, nghiệm thu đề tài NCKH Sinh viên năm 2026.